

Rapport d'analyse : Génération et visualisation de distributions statistiques

Groupe 3

PINCAUD Steve, SAVIARD Matthieu et CHARONNET Henri

6 mai 2026

Résumé

Ce rapport présente une implémentation en Python pour générer et visualiser les distributions statistiques suivantes : uniforme entière, uniforme réelle, normale, exponentielle et Poisson. L'objectif est d'analyser le comportement de ces distributions en fonction de la taille de l'échantillon, à l'aide d'histogrammes.

Table des matières

1 Introduction

Ce TP vise à modéliser et visualiser des distributions statistiques fondamentales en utilisant les bibliothèques `numpy` et `matplotlib`. L'implémentation repose sur une architecture orientée objet, avec une classe de base `Distribution` et des sous-classes pour chaque type de distribution. Les histogrammes générés permettent d'observer la convergence des échantillons vers les distributions théoriques lorsque la taille de l'échantillon augmente.

2 Prérequis

Pour exécuter ce code, les éléments suivants sont nécessaires :

- **Python 3.12** : Langage de programmation utilisé.
- **Bibliothèques** :
 - `numpy` : Pour la génération de nombres aléatoires et les opérations numériques.
 - `matplotlib` : Pour la visualisation des histogrammes.

- **Environnement** : Le code peut être exécuté dans n'importe quel environnement Python. La **seed** utilisée dans l'ensemble des exercices est `seed = 3`, correspondant au numéro du groupe. Cela garantit la reproductibilité des résultats : une même seed produit toujours la même séquence pseudo-aléatoire.

3 Première phase du TP

Pour **illustrer** notre travail lors de la première partie de ce TP, nous avons choisi de vous présenter une seule de nos implémentations.

3.1 Exercice 1

3.1.1 Objectif

Ce programme implémente un générateur congruentiel linéaire (GCL), un algorithme simple utilisé pour générer des suites de nombres pseudo-aléatoires. Il compare deux formules différentes et visualise les résultats.

3.1.2 Architecture du code

```

1 def gcl(x0, a, c, m, n=50):
2     x = x0
3     resultats = []
4     for _ in range(n):
5         x = (a * x + c) % m
6         resultats.append(x / m)
7     return resultats

```

Listing 1 – Implémentation de la fonction générateur congruentiel linéaire (gcl)

- **Paramètres** :

- `x0` : Valeur initiale (graine).
- `a`, `c`, `m` : Coefficients de la formule de récurrence.
- `n` : Nombre d'itérations (par défaut, 50).

- **Fonctionnement** :

- À chaque itération, calcule : $x = (a \times x + c) \bmod m$.
- Normalise les résultats en divisant par m pour obtenir des valeurs dans l'intervalle $[0, 1)$.
- Retourne une liste de n valeurs normalisées.

3.1.3 Formules testées

- **Formule 1** : $x_n = 3 \times x_{n-1} \bmod 150$
Appel : `gcl(x0=5, a=3, c=0, m=150)`
- **Formule 2** : $x_n = (5 \times x_{n-1} + 7) \bmod 200$
Appel : `gcl(x0=3, a=5, c=7, m=200)`

3.1.4 Visualisation

Utilise `matplotlib` pour afficher deux graphiques côte à côte :

- **Gauche** : Évolution des valeurs pour la Formule 1.
- **Droite** : Évolution des valeurs pour la Formule 2.

Chaque graphique montre les valeurs en fonction des itérations, avec des points reliés par des lignes ('o-').

3.1.5 Analyse et résultat

Les graphiques permettent de visualiser la périodicité des suites générées. Selon les paramètres `a`, `c`, `m`, les suites peuvent être plus ou moins aléatoires ou cycliques.

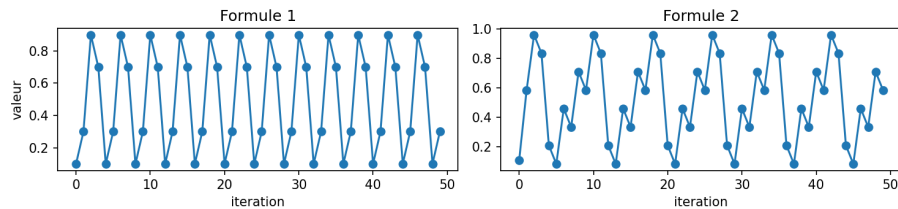


FIGURE 1 – Graphiques obtenus lors de l'exécution du code de l'exercice 1

3.2 Exercice 2

3.2.1 Objectif

Cet exercice vise à simuler une variable aléatoire discrète X suivant une loi de Bernoulli de paramètre $p = \frac{1}{3}$. L'objectif est de vérifier la convergence empirique de la proportion de succès ($X = 1$) vers la probabilité théorique $\frac{1}{3}$ lorsque la taille de l'échantillon n augmente.

3.2.2 Architecture du code

```
1 import numpy as np
2
3 rng = np.random.default_rng(seed=3)
4 for n in [100, 1000, 10000]:
5     U = rng.uniform(0, 1, size=n)
6     X = np.where(U < 1/3, 1, 2)
7     prop = np.mean(X == 1)
8     print(f"n={n:6d} : proportion X=1 = {prop:.4f} (th orique :
          0.3333) erreur = {abs(prop - 1/3):.4f}")
```

Listing 2 – Simulation d'une loi de Bernoulli avec p

3.2.3 Description du code

— Initialisation :

— `rng = np.random.default_rng(seed = 3)` :
Initialise un générateur de nombres aléatoires avec une graine fixe pour la reproductibilité.

— Boucle sur les tailles d'échantillon :

— Pour chaque n dans $[100, 1000, 10000]$:

- Génère n nombres uniformes dans $[0, 1)$ avec `rng.uniform(0, 1, size=n)`.
- Applique la règle : $X = 1$ si $U < \frac{1}{3}$, sinon $X = 2$.
- Calcule la proportion empirique de $X = 1$ avec `np.mean(X == 1)`.
- Affiche la proportion empirique, la valeur théorique ($\frac{1}{3}$), et l'erreur absolue entre les deux.

3.2.4 Analyse des résultats

- La proportion empirique de $X = 1$ converge vers $\frac{1}{3}$ lorsque n augmente.
- L'erreur absolue entre la proportion empirique et la valeur théorique diminue avec l'augmentation de n , illustrant la **loi des grands nombres**.
- Exemple de sortie attendue :
n= 100 : proportion X=1 = 0.3500 (théorique : 0.3333) erreur = 0.0167
n= 1000 : proportion X=1 = 0.3340 (théorique : 0.3333) erreur = 0.0007
n= 10000 : proportion X=1 = 0.3339 (théorique : 0.3333) erreur = 0.0006

4 Seconde phase du TP

4.1 Architecture du code

```
1 class Distribution:
2     """Classe de base représentant une distribution statistique."""
3
4     def __init__(self, name: str, rng: np.random.Generator):
5         self.name = name
6         self.rng = rng
7
8     def generate(self, n: int) -> np.ndarray:
9         raise NotImplementedError("La méthode generate() doit
10            être implémentée dans la sous-classe.")
11
12     def __repr__(self):
13         return f"{self.__class__.__name__}(name={self.name!r})"
```

Listing 3 – Implémentation de la classe Distribution mère

La classe `Distribution` est une classe abstraite qui définit l'interface commune à toutes les distributions. Elle contient :

- Un attribut `name` pour identifier la distribution.
- Un générateur aléatoire `rng` de type `numpy.random.Generator`.
- Une méthode abstraite `generate(n)` qui doit être implémentée par chaque sous-classe pour générer un échantillon de taille `n`.

Sous-classes de distributions Chaque sous-classe hérite de `Distribution` et implémente la méthode `generate(n)` selon la distribution spécifique :

- `UniformeEntiere` : Distribution uniforme discrète sur l'intervalle `[low, high]`. Utilise `rng.integers`.

```

1 class UniformeEntiere(Distribution):
2     """Distribution uniforme discrète sur [low, high]."""
3
4     def __init__(self, rng, low: int = 20, high: int = 40):
5         super().__init__(f"Uniforme entière ({low} {high})",
6             , rng)
7         self.low = low
8         self.high = high
9
10    def generate(self, n: int) -> np.ndarray:
11        return self.rng.integers(self.low, self.high + 1, size
    =n)

```

Listing 4 – Implémentation de la classe `UniformeEntiere`

- `UniformeReelle` : Distribution uniforme continue sur l'intervalle `[low, high)`. Utilise `rng.uniform`.

```

1 class UniformeReelle(Distribution):
2     """Distribution uniforme continue sur [low, high)."""
3
4     def __init__(self, rng, low: float = 2.0, high: float =
5         3.0):
6         super().__init__(f"Uniforme réelle ({low} {high})",
7             rng)
8         self.low = low
9         self.high = high
10
11    def generate(self, n: int) -> np.ndarray:
12        return self.rng.uniform(self.low, self.high, size=n)

```

Listing 5 – Implémentation de la classe `UniformeReelle`

- `Normale` : Distribution normale (gaussienne) avec moyenne `mu` et écart-type `sigma`. Utilise `rng.normal`.

```

1 class Normale(Distribution):
2     """Distribution normale (gaussienne)."""
3
4     def __init__(self, rng, mu: float = 0, sigma: float = 0.5)
5         :

```

```

5         super().__init__(f"Normale ( mu={mu}, sigma={sigma})", rng
6     )
7     self.mu = mu
8     self.sigma = sigma
9
10    def generate(self, n: int) -> np.ndarray:
11        return self.rng.normal(self.mu, self.sigma, size=n)

```

Listing 6 – Implémentation de la classe Normale

- **Exponentielle** : Distribution exponentielle avec paramètre d'échelle `scale`. Utilise `rng.exponential`.

```

1 class Exponentielle(Distribution):
2     """Distribution exponentielle."""
3
4     def __init__(self, rng, scale: float = 4):
5         super().__init__(f"Exponentielle (moy={scale})", rng)
6         self.scale = scale
7
8     def generate(self, n: int) -> np.ndarray:
9         return self.rng.exponential(self.scale, size=n)
10

```

Listing 7 – Implémentation de la classe Exponentielle

- **Poisson** : Distribution de Poisson avec paramètre `lam` (). Utilise `rng.poisson`.

```

1 class Poisson(Distribution):
2     """Distribution de Poisson."""
3
4     def __init__(self, rng, lam: float = 42):
5         super().__init__(f"Poisson ( lam={lam})", rng)
6         self.lam = lam
7
8     def generate(self, n: int) -> np.ndarray:
9         return self.rng.poisson(self.lam, size=n)
10

```

Listing 8 – Implémentation de la classe Poisson

Classe HistogrammeVisualiser Cette classe permet de générer et sauvegarder les histogrammes pour une liste de distributions et une liste de tailles d'échantillons `ns`. Elle propose une méthode `tracer()` qui :

- Crée une figure avec un sous-graphe par taille d'échantillon.
- Génère un histogramme pour chaque distribution et chaque taille d'échantillon.
- Sauvegarde les figures au format PNG (optionnel).
- Affiche les figures à l'écran (optionnel).

```

1 class HistogrammeVisualiser:
2     """Génère et sauvegarde les histogrammes pour une liste de
    distributions."""

```

```

3
4     def __init__(self, distributions: list[Distribution], ns: list[
5         int]):
6         self.distributions = distributions
7         self.ns = ns
8
9     def tracer(self, save: bool = True, show: bool = True):
10        for dist in self.distributions:
11            fig, axes = plt.subplots(1, len(self.ns), figsize=(16,
12            3))
13            fig.suptitle(dist.name, fontsize=13)
14
15            for ax, n in zip(axes, self.ns):
16                data = dist.generate(n)
17                ax.hist(data, bins='auto', edgecolor='white',
18                linewidth=0.4)
19                ax.set_title(f"n = {n}")
20                ax.set_xlabel("Valeur")
21                ax.set_ylabel("Fr quence")
22
23            plt.tight_layout()
24
25            if save:
26                filename = f"hist_{dist.name[:10].strip().encode('
27                utf-8').hex()}.png"
28                plt.savefig(filename, dpi=150)
29
30            if show:
31                plt.show()
32
33            plt.close(fig)

```

Listing 9 – Implémentation de la classe HistogrammeVisualiser

4.2 Analyse des résultats

4.2.1 Comportement des distributions

Pour chaque distribution, nous avons généré des histogrammes pour des tailles d'échantillon $n = 10, 100, 1000, 10000$. Voici les observations pour chaque type de distribution :

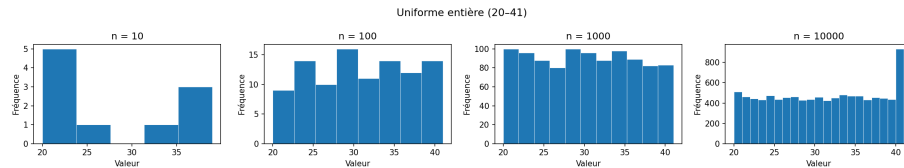


FIGURE 2 – Distribution uniforme entière (20–40)

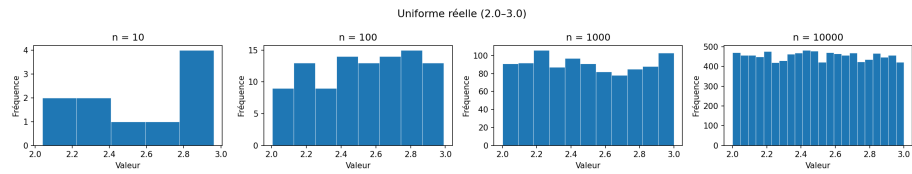


FIGURE 3 – Distribution uniforme réelle (2.0–3.0)

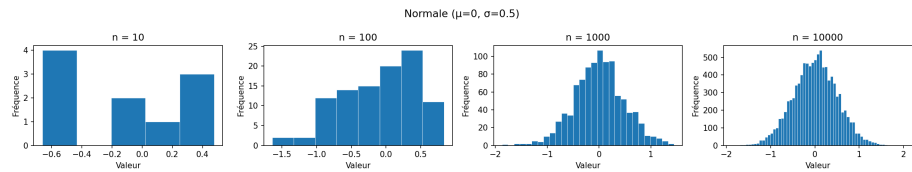


FIGURE 4 – Distribution normale ($\mu=0, \sigma=0.5$)

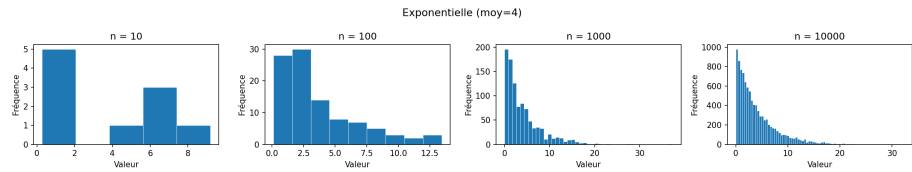


FIGURE 5 – Distribution exponentielle (moy=4)

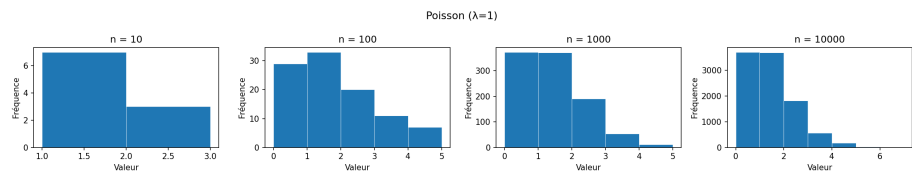


FIGURE 6 – Distribution de Poisson ($\lambda=1$)

— **Uniforme entière (20–40) :**

- Pour $n = 10$, l'histogramme est très irrégulier en raison du faible nombre d'échantillons.
- À partir de $n = 1000$, la distribution devient uniforme, avec une fréquence presque constante pour chaque valeur entière entre 20 et 40.
- Pour $n = 10000$, l'histogramme est parfaitement plat, confirmant la propriété d'équiprobabilité de la distribution uniforme discrète.

— **Uniforme réelle (2.0–3.0) :**

- Pour $n = 10$, l'histogramme est très bruité.

- Dès $n = 1000$, la distribution devient visuellement uniforme sur l'intervalle $[2.0, 3.0)$.
- Pour $n = 10000$, la densité est parfaitement constante, validant la propriété théorique de la distribution uniforme continue.
- **Normale ($\mu=0, \sigma=0.5$) :**
 - Même pour $n = 100$, la forme en cloche caractéristique de la distribution normale commence à apparaître.
 - Pour $n = 1000$ et $n = 10000$, l'histogramme se rapproche de plus en plus de la courbe théorique de la loi normale, centrée en 0 avec un écart-type de 0.5.
 - La convergence est rapide, grâce à la nature symétrique et concentrée de cette distribution.
- **Exponentielle ($\mu=4$) :**
 - Pour $n = 10$, l'histogramme est très irrégulier, avec des valeurs étalées.
 - À partir de $n = 1000$, la décroissance exponentielle caractéristique devient visible, avec une concentration de valeurs proches de 0.
 - Pour $n = 10000$, l'histogramme épouse parfaitement la forme théorique $f(x) = \frac{1}{4}e^{-x/4}$, confirmant la propriété de mémoire sans vieillissement de la loi exponentielle.
- **Poisson ($\mu=42$) :**
 - Pour $n = 10$, l'histogramme est très dispersé et peu informatif.
 - Dès $n = 1000$, une forme en cloche asymétrique apparaît, centrée autour de $\lambda = 42$.
 - Pour $n = 10000$, l'histogramme se rapproche de la distribution théorique de Poisson, avec un pic à 42 et une queue vers les valeurs supérieures.
 - La variance empirique se rapproche de la moyenne (42), comme attendu pour une loi de Poisson.

4.2.2 Convergence et loi des grands nombres

- Pour toutes les distributions, on observe une convergence des histogrammes vers leur forme théorique lorsque n augmente.
- Ce comportement illustre la loi des grands nombres, qui stipule que la moyenne empirique d'un échantillon converge vers l'espérance théorique lorsque la taille de l'échantillon tend vers l'infini.
- Les histogrammes pour $n = 10000$ sont très proches des densités de probabilité théoriques, validant ainsi l'implémentation des générateurs aléatoires.
- Les écarts observés pour $n = 10$ ou $n = 100$ sont normaux et reflètent la variabilité d'échantillonnage, qui diminue avec l'augmentation de n .

5 Annexes

5.0.1 Exercice 1

```

1 import matplotlib.pyplot as plt
2
3 def gcl(x0, a, c, m, n=50):
4     x = x0
5     resultats = []
6     for _ in range(n):
7         x = (a * x + c) % m
8         resultats.append(x / m)
9     return resultats
10
11 f1 = gcl(x0=5, a=3, c=0, m=150)
12 f2 = gcl(x0=3, a=5, c=7, m=200)
13
14 plt.figure(figsize=(12, 4))
15
16 plt.subplot(1, 2, 1)
17 plt.plot(f1, 'o-')
18 plt.title("Formule 1")
19 plt.xlabel("iteration")
20 plt.ylabel("valeur")
21
22 plt.subplot(1, 2, 2)
23 plt.plot(f2, 'o-')
24 plt.title("Formule 2")
25 plt.xlabel("iteration")
26
27 plt.tight_layout()
28 plt.show()

```

Listing 10 – Code complet de l'exercice 1

5.0.2 Exercice 2

```

1 import numpy as np
2
3 rng = np.random.default_rng(seed=3)
4
5 for n in [100, 1000, 10000]:
6     U = rng.uniform(0, 1, size=n)
7     X = np.where(U < 1/3, 1, 2)
8     prop = np.mean(X == 1)
9     print(f"n={n:6d} : proportion X=1 = {prop:.4f} (th orique :
10         0.3333) erreur = {abs(prop - 1/3):.4f}")

```

Listing 11 – Code complet de l'exercice 2

5.0.3 Phase de groupe

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 class Distribution:
5     """Classe de base repr sentant une distribution statistique."""
6     "
7
8     def __init__(self, name: str, rng: np.random.Generator):

```

```

8         self.name = name
9         self.rng = rng
10
11     def generate(self, n: int) -> np.ndarray:
12         raise NotImplementedError("La méthode generate() doit
13         être implémentée dans la sous-classe.")
14
15     def __repr__(self):
16         return f"{self.__class__.__name__}(name={self.name!r})"
17
18 class UniformeEntiere(Distribution):
19     """Distribution uniforme discrète sur [MIN, MAX]."""
20
21     def __init__(self, rng, low: int = 20, high: int = 41):
22         super().__init__(f"Uniforme entière ({low} {high})", rng)
23
24         self.low = low
25         self.high = high
26
27     def generate(self, n: int) -> np.ndarray:
28         return self.rng.integers(self.low, self.high + 1, size=n)
29
30 class UniformeReelle(Distribution):
31     """Distribution uniforme continue sur [MIN, MAX]."""
32
33     def __init__(self, rng, low: float = 2.0, high: float = 3.0):
34         super().__init__(f"Uniforme réelle ({low} {high})", rng)
35
36         self.low = low
37         self.high = high
38
39     def generate(self, n: int) -> np.ndarray:
40         return self.rng.uniform(self.low, self.high, size=n)
41
42 class Normale(Distribution):
43     """Distribution normale (gaussienne)."""
44
45     def __init__(self, rng, mu: float = 0, sigma: float = 0.5):
46         super().__init__(f"Normale (μ={mu}, σ={sigma})", rng)
47
48         self.mu = mu
49         self.sigma = sigma
50
51     def generate(self, n: int) -> np.ndarray:
52         return self.rng.normal(self.mu, self.sigma, size=n)
53
54 class Exponentielle(Distribution):
55     """Distribution exponentielle."""
56
57     def __init__(self, rng, scale: float = 4):
58         super().__init__(f"Exponentielle (moy={scale})", rng)
59
60         self.scale = scale
61
62     def generate(self, n: int) -> np.ndarray:
63         return self.rng.exponential(self.scale, size=n)
64
65 class Poisson(Distribution):
66     """Distribution de Poisson."""

```

```

63     def __init__(self, rng, lam: float = 42):
64         super().__init__(f"Poisson (={lam})", rng)
65         self.lam = lam
66
67     def generate(self, n: int) -> np.ndarray:
68         return self.rng.poisson(self.lam, size=n)
69
70 class HistogrammeVisualiser:
71     """Gère et sauvegarde les histogrammes pour une liste de
72     distributions."""
73
74     def __init__(self, distributions: list[Distribution], ns: list[
75         int]):
76         self.distributions = distributions
77         self.ns = ns
78
79     def tracer(self, save: bool = True, show: bool = True):
80         for dist in self.distributions:
81             fig, axes = plt.subplots(1, len(self.ns), figsize=(16,
82             3))
83             fig.suptitle(dist.name, fontsize=13)
84
85             for ax, n in zip(axes, self.ns):
86                 data = dist.generate(n)
87                 ax.hist(data, bins='auto', edgecolor='white',
88                 linewidth=0.4)
89                 ax.set_title(f"n = {n}")
90                 ax.set_xlabel("Valeur")
91                 ax.set_ylabel("Fréquence")
92
93             plt.tight_layout()
94
95             if save:
96                 filename = f"hist_{dist.name[:10].strip().encode('
97                 utf-8').hex()}.png"
98                 plt.savefig(filename, dpi=150)
99
100             if show:
101                 plt.show()
102
103             plt.close(fig)
104
105 if __name__ == "__main__":
106     print("+++ START TP1 Main ...")
107     rng = np.random.default_rng(seed=3)
108
109     distributions = [
110         UniformeEntiere(rng),
111         UniformeReelle(rng),
112         Normale(rng),
113         Exponentielle(rng),
114         Poisson(rng),
115     ]
116
117     ns = [10, 100, 1000, 10000]
118
119     visualiseur = HistogrammeVisualiser(distributions, ns)

```

```
115     visualiseur.tracer(save=True, show=True)
116
117     print("+++ END TP1 Main.")
```

Listing 12 – Code complet du TP de groupe